7 Creating REST services

This chapter covers

- Defining REST endpoints in Spring MVC
- Automatic repository-based REST endpoints
- Consuming REST APIs

“The web browser is dead. What now?”

Several years ago, I heard someone suggest that the web browser was nearing legacy status and that something else would take over. But how could this be? What could possibly dethrone the near-ubiquitous web browser? How would we consume the growing number of sites and online services if not with a web browser? Surely these were the ramblings of a madman!

Fast-forward to the present day, and it’s clear that the web browser hasn’t gone away. But it no longer reigns as the primary means of accessing the internet. Mobile devices, tablets, smart watches, and voice-based devices are now commonplace. And even many browser-based applications are actually running JavaScript applications rather than letting the browser be a dumb terminal for server-rendered content.

With such a vast selection of client-side options, many applications have adopted a common design where the user interface is pushed closer to the client and the server exposes an API through which all kinds of clients can interact with the backend functionality.

In this chapter, you’re going to use Spring to provide a REST API for the Taco Cloud application. You’ll use what you learned about Spring MVC in chapter 2 to create RESTful endpoints with Spring MVC controllers. You’ll also automatically expose REST endpoints for the Spring Data repositories you defined in chapters 3 and 4. Finally, we’ll look at ways to test and secure those endpoints.

But first, you’ll start by writing a few new Spring MVC controllers that expose backend functionality with REST endpoints to be consumed by a rich web frontend.

7.1 Writing RESTful controllers

In a nutshell, REST APIs aren't much different from websites. Both involve responding to HTTP requests. But the key difference is that instead of responding to those requests with HTML, as websites do, REST APIs typically respond with a data-oriented format such as JSON or XML.

In chapter 2 you used `@GetMapping` and `@PostMapping` annotations to fetch and post data to the server. Those same annotations will still come in handy as you define your REST API. In addition, Spring MVC supports a handful of other annotations for various types of HTTP requests, as listed in table 7.1.

Table 7.1 Spring MVC's HTTP request-handling annotations

Annotation	HTTP method	Typical use[1]
@GetMapping	HTTP GET requests	Reading resource data
`@PostMapping`	HTTP `POST` requests	Creating a resource
@PutMapping	HTTP PUT requests	Updating a resource
`@PatchMapping`	HTTP `PATCH` requests	Updating a resource
@DeleteMapping	HTTP DELETE requests	Deleting a resource
`@RequestMapping`	General-purpose request handling; HTTP `method` specified in the method attribute	

To see these annotations in action, you'll start by creating a simple REST endpoint that fetches a few of the most recently created tacos.

1.1 Retrieving data from the server

One thing that we'd like the Taco Cloud application to be able to do is allow taco fanatics to design their own taco creations and share them with their follow taco lovers. One way to do that is to display a list of the most recently created tacos on the website.

In support of that feature, we need to create an endpoint that handles `GET` requests for /api/tacos which include a "recent" parameter and responds with a list of recently designed tacos. You'll create a new controller to handle such a request. The next listing shows the controller for the job.

Listing 7.1 A RESTful controller for taco design API requests

```
package tacos.web.api;

import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Sort;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import tacos.Taco;
import tacos.data.TacoRepository;

@RestController
@RequestMapping(path="/api/tacos",                    (1)
                produces="application/json")
@CrossOrigin(origins="http://tacocloud:8080")         (2)
public class TacoController {
  private TacoRepository tacoRepo;

  public TacoController(TacoRepository tacoRepo) {
    this.tacoRepo = tacoRepo;
  }

  @GetMapping(params="recent")
  public Iterable<Taco> recentTacos() {               (3)
    PageRequest page = PageRequest.of(
          0, 12, Sort.by("createdAt").descending());
    return tacoRepo.findAll(page).getContent();
  }
}
```

❶ Handles requests for /api/tacos

❷ Allows cross-origin requests

❸ Fetches and returns recent taco designs

You may be thinking that this controller's name sounds somewhat familiar. In chapter 2 you created a similarly named `DesignTacoController` that handled similar types of requests. But where that controller was for producing an HTML result in the Taco Cloud application, this new `TacoController` is a REST controller, as indicated by the `@RestController` annotation.

The `@RestController` annotation serves two purposes. First, it's a stereotype annotation like `@Controller` and `@Service` that marks a class for discovery by component scanning. But most relevant to the discussion of REST, the `@RestController` annotation tells Spring that all handler methods in the controller should have their return value written directly to the body of the response, rather than being carried in the model to a view for rendering.

Alternatively, you could have annotated `TacoController` with `@Controller`, just like any Spring MVC controller. But then you'd need to also annotate all of the handler methods with `@ResponseBody` to achieve the same result. Yet another option would be to return a `ResponseEntity` object, which we'll discuss in a moment.

The `@RequestMapping` annotation at the class level works with the `@GetMapping` annotation on the `recentTacos()` method to specify that the `recentTacos()` method is responsible for handling `GET` requests for /api/tacos?recent.

You'll notice that the `@RequestMapping` annotation also sets a `produces` attribute. This specifies that any of the handler methods in `TacoController` will handle requests only if the client sends a request with an `Accept` header that includes `"application/json"`, indicating that the client can handle responses only in JSON format. This use of `produces` limits your API to only producing JSON results, and it allows for another controller (perhaps the `TacoController` from chapter 2) to handle requests with the same paths, so long as those requests don't require JSON output.

Even though setting `produces` to `"application/json"` limits your API to being JSON-based (which is fine for your needs), you're welcome to set `produces` to an array of `String` for multiple content types. For example, to allow for XML output, you could add `"text/xml"` to the `produces` attribute as follows:

```
@RequestMapping(path="/api/tacos",
                produces={"application/json", "text/xml"})
```

The other thing you may have noticed in listing 7.1 is that the class is annotated with `@CrossOrigin`. It's common for a JavaScript-based user interface, such as those written in a framework like Angular or ReactJS, to be served from a separate host and/or port from the API (at least for now), and the web browser will prevent your client from consuming the API. This restriction can be overcome by including CORS (cross-origin resource sharing) headers in the server responses. Spring makes it easy to apply CORS with the `@CrossOrigin` annotation.

As applied here, `@CrossOrigin` allows clients from localhost, port 8080, to access the API. The `origins` attribute accepts an array, however, so you can also specify multiple values, as shown next:

```
@RestController
@RequestMapping(path="/api/tacos",
                produces="application/json")
@CrossOrigin(origins={"http://tacocloud:8080", "http://tacocloud.com"})
public class TacoController {
  ...
}
```

The logic within the `recentTacos()` method is fairly straightforward. It constructs a `PageRequest` object that specifies that you want only the first (0th) page of 12 results, sorted in descending order by the taco's creation date. In short, you want a dozen of the most recently created taco designs. The `PageRequest` is passed into the call to the `findAll()` method of `TacoRepository`, and the content of that page of results is returned to the client (which, as you saw in listing 7.1, will be used as model data to display to the user).

You now have the start of a Taco Cloud API for your client. For development testing purposes, you may also want to use command-line utilities like `curl` or HTTPie (**https://httpie.org/**) to poke about the API. For example, the following command line shows how you might fetch recently created tacos with `curl`:

```
$ curl localhost:8080/api/tacos?recent
```

Or like this, if you prefer HTTPie:

```
$ http :8080/api/tacos?recent
```

Initially, the database will be empty, so the results from these requests will likewise be empty. We'll see in a moment how to handle `POST` requests that save tacos. But in the meantime, you could add an `CommandLineRunner` bean to preload the database with some test data. The following `CommandLineRunner` bean method shows how you might preload a few ingredients and a few tacos:

```
@Bean
public CommandLineRunner dataLoader(
    IngredientRepository repo,
    UserRepository userRepo,
    PasswordEncoder encoder,
    TacoRepository tacoRepo) {
  return args -> {
    Ingredient flourTortilla = new Ingredient(
        "FLTO", "Flour Tortilla", Type.WRAP);
    Ingredient cornTortilla = new Ingredient(
        "COTO", "Corn Tortilla", Type.WRAP);
    Ingredient groundBeef = new Ingredient(
        "GRBF", "Ground Beef", Type.PROTEIN);
    Ingredient carnitas = new Ingredient(
        "CARN", "Carnitas", Type.PROTEIN);
    Ingredient tomatoes = new Ingredient(
        "TMTO", "Diced Tomatoes", Type.VEGGIES);
    Ingredient lettuce = new Ingredient(
        "LETC", "Lettuce", Type.VEGGIES);
    Ingredient cheddar = new Ingredient(
        "CHED", "Cheddar", Type.CHEESE);
    Ingredient jack = new Ingredient(
        "JACK", "Monterrey Jack", Type.CHEESE);
    Ingredient salsa = new Ingredient(
        "SLSA", "Salsa", Type.SAUCE);
    Ingredient sourCream = new Ingredient(
        "SRCR", "Sour Cream", Type.SAUCE);
    repo.save(flourTortilla);
    repo.save(cornTortilla);
    repo.save(groundBeef);
    repo.save(carnitas);
    repo.save(tomatoes);
    repo.save(lettuce);
    repo.save(cheddar);
    repo.save(jack);
    repo.save(salsa);
```

```
    repo.save(sourCream);

    Taco taco1 = new Taco();
    taco1.setName("Carnivore");
    taco1.setIngredients(Arrays.asList(
          flourTortilla, groundBeef, carnitas,
          sourCream, salsa, cheddar));
    tacoRepo.save(taco1);

    Taco taco2 = new Taco();
    taco2.setName("Bovine Bounty");
    taco2.setIngredients(Arrays.asList(
          cornTortilla, groundBeef, cheddar,
          jack, sourCream));
    tacoRepo.save(taco2);

    Taco taco3 = new Taco();
    taco3.setName("Veg-Out");
    taco3.setIngredients(Arrays.asList(
          flourTortilla, cornTortilla, tomatoes,
          lettuce, salsa));
    tacoRepo.save(taco3);
  };
}
```

Now if you try to use `curl` or HTTPie to make a request to the recent tacos endpoint, you'll get a response something like this (response formatted for readability):

```
$ curl localhost:8080/api/tacos?recent
[
  {
    "id": 4,
    "name": "Veg-Out",
    "createdAt": "2021-08-02T00:47:09.624+00:00",
    "ingredients": [
      { "id": "FLTO", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "COTO", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "TMTO", "name": "Diced Tomatoes", "type": "VEGGIES" },
      { "id": "LETC", "name": "Lettuce", "type": "VEGGIES" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" }
    ]
  },
  {
    "id": 3,
    "name": "Bovine Bounty",
```

```
    "createdAt": "2021-08-02T00:47:09.621+00:00",
    "ingredients": [
      { "id": "COTO", "name": "Corn Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" },
      { "id": "JACK", "name": "Monterrey Jack", "type": "CHEESE" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" }
    ]
  },
  {
    "id": 2,
    "name": "Carnivore",
    "createdAt": "2021-08-02T00:47:09.520+00:00",
    "ingredients": [
      { "id": "FLTO", "name": "Flour Tortilla", "type": "WRAP" },
      { "id": "GRBF", "name": "Ground Beef", "type": "PROTEIN" },
      { "id": "CARN", "name": "Carnitas", "type": "PROTEIN" },
      { "id": "SRCR", "name": "Sour Cream", "type": "SAUCE" },
      { "id": "SLSA", "name": "Salsa", "type": "SAUCE" },
      { "id": "CHED", "name": "Cheddar", "type": "CHEESE" }
    ]
  }
]
```


Now let's say that you want to offer an endpoint that fetches a single taco by its ID. By using a placeholder variable in the handler method's path and accepting a path variable, you can capture the ID and use it to look up the `Taco` object through the repository as follows:

```
@GetMapping("/{id}")
public Optional<Taco> tacoById(@PathVariable("id") Long id) {
  return tacoRepo.findById(id);
}
```

Because the controller's base path is /api/tacos, this controller method handles `GET` requests for /api/tacos/{id}, where the {id} portion of the path is a placeholder. The actual value in the request is given to the `id` parameter, which is mapped to the `{id}` placeholder by `@PathVariable`.

Inside of `tacoById()`, the `id` parameter is passed to the repository's `findById()` method to fetch the `Taco`. The repository's `findById()` method returns an `Optional <Taco>`, be-

cause it is possible that there may not be a taco that matches the given ID. The `Optional<Taco>` is simply returned from the controller method.

Spring then takes the `Optional<Taco>` and calls its `get()` method to produce the response. If the ID doesn't match any known tacos, the response body will contain "null" and the response's HTTP status code will be 200 (OK). The client is handed a response it can't use, but the status code indicates everything is fine. A better approach would be to return a response with an HTTP 404 (NOT FOUND) status.

As it's currently written, there's no easy way to return a 404 status code from `tacoById()`. But if you make a few small tweaks, you can set the status code appropriately, as shown here:

```
@GetMapping("/{id}")
public ResponseEntity<Taco> tacoById(@PathVariable("id") Long id) {
  Optional<Taco> optTaco = tacoRepo.findById(id);
  if (optTaco.isPresent()) {
    return new ResponseEntity<>(optTaco.get(), HttpStatus.OK);
  }
  return new ResponseEntity<>(null, HttpStatus.NOT_FOUND);
}
```

Now, instead of returning a `Taco` object, `tacoById()` returns a `ResponseEntity<Taco>`. If the taco is found, you wrap the `Taco` object in a `ResponseEntity` with an HTTP status of OK (which is what the behavior was before). But if the taco isn't found, you wrap a `null` in a `ResponseEntity` along with an HTTP status of NOT FOUND to indicate that the client is trying to fetch a taco that doesn't exist.

Defining an endpoint that returns information is only the start. What if your API needs to receive data from the client? Let's see how you can write controller methods that handle input on the requests.

1.2 Sending data to the server

So far your API is able to return up to a dozen of the most recently created tacos. But how do those tacos get created in the first place?

Although you could use a `CommandLineRunner` bean to preload the database with some test taco data, ultimately taco data will come from users when they craft their taco creations. Therefore, we'll need to write a method in `TacoController` that handles requests

containing taco designs and save them to the database. By adding the following `postTaco()` method to `TacoController`, you enable the controller to do exactly that:

```
@PostMapping(consumes="application/json")
@ResponseStatus(HttpStatus.CREATED)
public Taco postTaco(@RequestBody Taco taco) {
  return tacoRepo.save(taco);
}
```

Because `postTaco()` will handle an HTTP `POST` request, it's annotated with `@PostMapping` instead of `@GetMapping`. You're not specifying a `path` attribute here, so the `postTaco()` method will handle requests for /api/tacos as specified in the class-level `@RequestMapping` on `TacoController`.

You do set the `consumes` attribute, however. The `consumes` attribute is to request input what `produces` is to request output. Here you use `consumes` to say that the method will only handle requests whose `Content-type` matches `application/json`.

The method's `Taco` parameter is annotated with `@RequestBody` to indicate that the body of the request should be converted to a `Taco` object and bound to the parameter. This annotation is important—without it, Spring MVC would assume that you want request parameters (either query parameters or form parameters) to be bound to the `Taco` object. But the `@RequestBody` annotation ensures that JSON in the request body is bound to the `Taco` object instead.

Once `postTaco()` has received the `Taco` object, it passes it to the `save()` method on the `TacoRepository`.

You may have also noticed that I've annotated the `postTaco()` method with `@ResponseStatus(HttpStatus.CREATED)`. Under normal circumstances (when no exceptions are thrown), all responses will have an HTTP status code of 200 (OK), indicating that the request was successful. Although an HTTP 200 response is always welcome, it's not always descriptive enough. In the case of a `POST` request, an HTTP status of 201 (CREATED) is more descriptive. It tells the client that not only was the request successful but a resource was created as a result. It's always a good idea to use `@ResponseStatus` where appropriate to communicate the most descriptive and accurate HTTP status code to the client.

Although you've used `@PostMapping` to create a new `Taco` resource, `POST` requests can also be used to update resources. Even so, `POST` requests are typically used for re-

source creation, and `PUT` and `PATCH` requests are used to update resources. Let's see how you can update data using `@PutMapping` and `@PatchMapping`.

1.3 Updating data on the server

Before you write any controller code for handling HTTP `PUT` or `PATCH` commands, you should take a moment to consider the elephant in the room: why are there two different HTTP methods for updating resources?

Although it's true that `PUT` is often used to update resource data, it's actually the semantic opposite of `GET`. Whereas `GET` requests are for transferring data from the server to the client, `PUT` requests are for sending data from the client to the server.

In that sense, `PUT` is really intended to perform a wholesale *replacement* operation rather than an update operation. In contrast, the purpose of HTTP `PATCH` is to perform a patch or partial update of resource data.

For example, suppose you want to be able to change the address on an order. One way we could achieve this through the REST API is with a `PUT` request handled like this:

```
@PutMapping(path="/{orderId}", consumes="application/json")
public TacoOrder putOrder(
                @PathVariable("orderId") Long orderId,
                @RequestBody TacoOrder order) {
  order.setId(orderId);
  return repo.save(order);
}
```

This could work, but it would require that the client submit the complete order data in the `PUT` request. Semantically, `PUT` means "put this data at this URL," essentially replacing any data that's already there. If any of the order's properties are omitted, that property's value would be overwritten with `null`. Even the tacos in the order would need to be set along with the order data or else they'd be removed from the order.

If `PUT` does a wholesale replacement of the resource data, then how should you handle requests to do just a partial update? That's what HTTP `PATCH` requests and Spring's `@PatchMapping` are good for. Here's how you might write a controller method to handle a `PATCH` request for an order:

```
@PatchMapping(path="/{orderId}", consumes="application/json")
public TacoOrder patchOrder(@PathVariable("orderId") Long orderId,
```

```
                @RequestBody TacoOrder patch) {

    TacoOrder order = repo.findById(orderId).get();
    if (patch.getDeliveryName() != null) {
      order.setDeliveryName(patch.getDeliveryName());
    }
    if (patch.getDeliveryStreet() != null) {
      order.setDeliveryStreet(patch.getDeliveryStreet());
    }
    if (patch.getDeliveryCity() != null) {
      order.setDeliveryCity(patch.getDeliveryCity());
    }
    if (patch.getDeliveryState() != null) {
      order.setDeliveryState(patch.getDeliveryState());
    }
    if (patch.getDeliveryZip() != null) {
      order.setDeliveryZip(patch.getDeliveryZip());
    }
    if (patch.getCcNumber() != null) {
      order.setCcNumber(patch.getCcNumber());
    }
    if (patch.getCcExpiration() != null) {
      order.setCcExpiration(patch.getCcExpiration());
    }
    if (patch.getCcCVV() != null) {
      order.setCcCVV(patch.getCcCVV());
    }
    return repo.save(order);
  }
```

The first thing to note here is that the `patchOrder()` method is annotated with `@PatchMapping` instead of `@PutMapping`, indicating that it should handle HTTP `PATCH` requests instead of `PUT` requests.

But the one thing you've no doubt noticed is that the `patchOrder()` method is a bit more involved than the `putOrder()` method. That's because Spring MVC's mapping annotations, including `@PatchMapping` and `@PutMapping`, specify only what kinds of requests a method should handle. These annotations don't dictate how the request will be handled. Even though `PATCH` semantically implies a partial update, it's up to you to write code in the handler method that actually performs such an update.

In the case of the `putOrder()` method, you accepted the complete data for an order and saved it, adhering to the semantics of HTTP `PUT`. But in order for `patchMapping()` to

adhere to the semantics of HTTP `PATCH`, the body of the method requires more intelligence. Instead of completely replacing the order with the new data sent in, it inspects each field of the incoming `TacoOrder` object and applies any non-`null` values to the existing order. This approach allows the client to send only the properties that should be changed and enables the server to retain existing data for any properties not specified by the client.

There's more than one way to PATCH

The patching approach applied in the `patchOrder()` method has the following limitations:

- If `null` values are meant to specify no change, how can the client indicate that a field should be set to `null`?
- There's no way of removing or adding a subset of items from a collection. If the client wants to add or remove an entry from a collection, it must send the complete altered collection.

There's really no hard-and-fast rule about how `PATCH` requests should be handled or what the incoming data should look like. Rather than sending the actual domain data, a client could send a patch-specific description of the changes to be applied. Of course, the request handler would have to be written to handle patch instructions instead of the domain data.

In both `@PutMapping` and `@PatchMapping`, notice that the request path references the resource that's to be changed. This is the same way paths are handled by `@GetMapping`-annotated methods.

You've now seen how to fetch and post resources with `@GetMapping` and `@PostMapping`. And you've seen two different ways of updating a resource with `@PutMapping` and `@PatchMapping`. All that's left is handling requests to delete a resource.

1.4 Deleting data from the server

Sometimes data simply isn't needed anymore. In those cases, a client should be able to request that a resource be removed with an HTTP `DELETE` request.

Spring MVC's `@DeleteMapping` comes in handy for declaring methods that handle `DELETE` requests. For example, let's say you want your API to allow for an order resource to be deleted. The following controller method should do the trick:

```
@DeleteMapping("/{orderId}")
@ResponseStatus(HttpStatus.NO_CONTENT)
public void deleteOrder(@PathVariable("orderId") Long orderId) {
  try {
    repo.deleteById(orderId);
  } catch (EmptyResultDataAccessException e) {}
}
```

By this point, the idea of another mapping annotation should be old hat to you. You've already seen `@GetMapping`, `@PostMapping`, `@PutMapping`, and `@PatchMapping`—each specifying that a method should handle requests for their corresponding HTTP methods. It will probably come as no surprise to you that `@DeleteMapping` is used to specify that the `deleteOrder()` method is responsible for handling `DELETE` requests for /orders/{orderId}.

The code within the method is what does the actual work of deleting an order. In this case, it takes the order ID, provided as a path variable in the URL, and passes it to the repository's `deleteById()` method. If the order exists when that method is called, it will be deleted. If the order doesn't exist, an `EmptyResultDataAccessException` will be thrown.

I've chosen to catch the `EmptyResultDataAccessException` and do nothing with it. My thinking here is that if you try to delete a resource that doesn't exist, the outcome is the same as if it did exist prior to deletion—that is, the resource will be nonexistent. Whether it existed before is irrelevant. Alternatively, I could've written `deleteOrder()` to return a `ResponseEntity`, setting the body to `null` and the HTTP status code to NOT FOUND.

The only other thing to take note of in the `deleteOrder()` method is that it's annotated with `@ResponseStatus` to ensure that the response's HTTP status is 204 (NO CONTENT). There's no need to communicate any resource data back to the client for a resource that no longer exists, so responses to `DELETE` requests typically have no body and, therefore, should communicate an HTTP status code to let the client know not to expect any content.

Your Taco Cloud API is starting to take shape. Now a client can be written to consume this API, presenting ingredients, accepting orders, and displaying recently created tacos. We'll talk about writing REST client code a little later in 7.3. But for now, let's see another way to create REST API endpoints: automatically based on Spring Data repositories.

7.2 Enabling data-backed services

As you saw in chapter 3, Spring Data performs a special kind of magic by automatically creating repository implementations based on interfaces you define in your code. But Spring Data has another trick up its sleeve that can help you define APIs for your application.

Spring Data REST is another member of the Spring Data family that automatically creates REST APIs for repositories created by Spring Data. By doing little more than adding Spring Data REST to your build, you get an API with operations for each repository interface you've defined.

To start using Spring Data REST, add the following dependency to your build:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-rest</artifactId>
</dependency>
```

Believe it or not, that's all that's required to expose a REST API in a project that's already using Spring Data for automatic repositories. By simply having the Spring Data REST starter in the build, the application gets autoconfiguration that enables automatic creation of a REST API for any repositories that were created by Spring Data (including Spring Data JPA, Spring Data Mongo, and so on).

The REST endpoints that Spring Data REST creates are at least as good as (and possibly even better than) the ones you've created yourself. So at this point, feel free to do a little demolition work and remove any `@RestController`-annotated classes you've created up to this point before moving on.

To try out the endpoints provided by Spring Data REST, you can fire up the application and start poking at some of the URLs. Based on the set of repositories you've already defined for Taco Cloud, you should be able to perform `GET` requests for tacos, ingredients, orders, and users.

For example, you can get a list of all ingredients by making a `GET` request for /ingredients. Using `curl`, you might get something that looks like this (abridged to show only the first ingredient):

```
$ curl localhost:8080/ingredients
{
  "_embedded" : {
    "ingredients" : [ {
      "name" : "Flour Tortilla",
      "type" : "WRAP",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/ingredients/FLTO"
        },
        "ingredient" : {
          "href" : "http://localhost:8080/ingredients/FLTO"
        }
      }
    },
    ...
    ]
  },
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients"
    },
    "profile" : {
      "href" : "http://localhost:8080/profile/ingredients"
    }
  }
}
```

Wow! By doing nothing more than adding a dependency to your build, you're not only getting an endpoint for ingredients, but the resources that come back also contain hyperlinks! These hyperlinks are implementations of Hypermedia as the Engine of Application State, or HATEOAS for short. A client consuming this API could (optionally) use these hyperlinks as a guide for navigating the API and performing the next request.

The Spring HATEOAS project (**https://spring.io/projects/spring-hateoas**) provides general support for adding hypermedia links in your Spring MVC controller responses. But Spring Data REST automatically adds these links in the responses to its generated APIs.

To HATEOAS or not to HATEOAS?

The general idea of HATEOAS is that it enables a client to navigate an API in much the same way that a human may navigate a website: by following links. Rather than encode

API details in a client and having the client construct URLs for every request, the client can select a link, by name, from the list of hyperlinks and use it to make their next request. In this way, the client doesn't need to be coded to know the structure of an API and can instead use the API itself as a roadmap through the API.

On the other hand, the hyperlinks do add a small amount of extra data in the payload and add some complexity requiring that the client know how to navigate using those hyperlinks. For this reason, API developers often forego the use of HATEOAS, and client developers often simply ignore the hyperlinks if there are any in an API.

Other than the free hyperlinks you get from Spring Data REST responses, we'll ignore HATEOAS and focus on simple, nonhypermedia APIs.

Pretending to be a client of this API, you can also use `curl` to follow the `self` link for the flour tortilla entry as follows:

```
$ curl http://localhost:8080/ingredients/FLTO
{
  "name" : "Flour Tortilla",
  "type" : "WRAP",
  "_links" : {
    "self" : {
      "href" : "http://localhost:8080/ingredients/FLTO"
    },
    "ingredient" : {
      "href" : "http://localhost:8080/ingredients/FLTO"
    }
  }
}
```

To avoid getting too distracted, we won't waste much more time in this book digging into each and every endpoint and option that Spring Data REST has created. But you should know that it also supports `POST`, `PUT`, and `DELETE` methods for the endpoints it creates. That's right: you can `POST` to /ingredients to create a new ingredient and `DELETE` /ingredients/FLTO to remove flour tortillas from the menu.

One thing you might want to do is set a base path for the API so that its endpoints are distinct and don't collide with any controllers you write. To adjust the base path for the API, set the `spring.data.rest.base-path` property as shown next:

```
spring:
  data:
    rest:
      base-path: /data-api
```

This sets the base path for Spring Data REST endpoints to /data-api. Although you can set the base path to anything you'd like, the choice of /data-api ensures that endpoints exposed by Spring Data REST don't collide with any other controllers, including those whose path begins with "/api" that we created earlier in this chapter. Consequently, the ingredients endpoint is now /data-api/ingredients. Now give this new base path a spin by requesting a list of tacos as follows:

```
$ curl http://localhost:8080/data-api/tacos
{
  "timestamp": "2018-02-11T16:22:12.381+0000",
  "status": 404,
  "error": "Not Found",
  "message": "No message available",
  "path": "/api/tacos"
}
```

Oh dear! That didn't work quite as expected. You have an `Ingredient` entity and an `IngredientRepository` interface, which Spring Data REST exposed with a /data-api/ingredients endpoint. So if you have a `Taco` entity and a `TacoRepository` interface, why doesn't Spring Data REST give you a /data-api/tacos endpoint?

2.1 Adjusting resource paths and relation names

Actually, Spring Data REST does give you an endpoint for working with tacos. But as clever as Spring Data REST can be, it shows itself to be a tiny bit less awesome in how it exposes the tacos endpoint.

When creating endpoints for Spring Data repositories, Spring Data REST tries to pluralize the associated entity class. For the `Ingredient` entity, the endpoint is /data-api/ingredients. For the `TacoOrder` entity, it's /data-api/orders. So far, so good.

But sometimes, such as with "taco," it trips up on a word and the pluralized version isn't quite right. As it turns out, Spring Data REST pluralized "taco" as "tacoes," so to make a request for tacos, you must play along and request /data-api/tacoes, as shown here:

```
$ curl localhost:8080/data-api/tacoes
{
  "_embedded" : {
    "tacoes" : [ {
      "name" : "Carnivore",
      "createdAt" : "2018-02-11T17:01:32.999+0000",
      "_links" : {
        "self" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
        "taco" : {
          "href" : "http://localhost:8080/data-api/tacoes/2"
        },
        "ingredients" : {
          "href" : "http://localhost:8080/data-api/tacoes/2/ingredients"
        }
      }
    }]
  },
  "page" : {
    "size" : 20,
    "totalElements" : 3,
    "totalPages" : 1,
    "number" : 0
  }
}
```

You may be wondering how I knew that "taco" would be mispluralized as "tacoes." As it turns out, Spring Data REST also exposes a home resource that lists links for all exposed endpoints. Just make a `GET` request to the API base path to get the goods as follows:

```
$ curl localhost:8080/api
{
  "_links" : {
    "orders" : {
      "href" : "http://localhost:8080/data-api/orders"
    },
    "ingredients" : {
      "href" : "http://localhost:8080/data-api/ingredients"
    },
    "tacoes" : {
      "href" : "http://localhost:8080/data-api/tacoes{?page,size,sort}",
      "templated" : true
    },
```

```
    "users" : {
      "href" : "http://localhost:8080/data-api/users"
    },
    "profile" : {
      "href" : "http://localhost:8080/data-api/profile"
    }
  }
}
```

As you can see, the home resource shows the links for all of your entities. Everything looks good, except for the `tacoes` link, where both the relation name and the URL have that odd pluralization of "taco."

The good news is that you don't have to accept this little quirk of Spring Data REST. By adding the following simple annotation to the `Taco` class, you can tweak both the relation name and that path:

```
@Data
@Entity
@RestResource(rel="tacos", path="tacos")
public class Taco {
  ...
}
```

The `@RestResource` annotation lets you give the entity any relation name and path you want. In this case, you're setting them both to `"tacos"`. Now when you request the home resource, you see the `tacos` link with correct pluralization, as shown next:

```
"tacos" : {
  "href" : "http://localhost:8080/data-api/tacos{?page,size,sort}",
  "templated" : true
},
```

This also sorts out the path for the endpoint so that you can issue requests against /data-api/tacos to work with taco resources.

Speaking of sorting things out, let's look at how you can sort the results from Spring Data REST endpoints.

2.2 Paging and sorting

You may have noticed that the links in the home resource all offer optional `page`, `size`, and `sort` parameters. By default, requests to a collection resource such as /data-api/tacos will return up to 20 items per page from the first page. But you can adjust the page size and the page displayed by specifying the `page` and `size` parameters in your request.

For example, to request the first page of tacos where the page size is `5`, you can issue the following `GET` request (using `curl`):

```
$ curl "localhost:8080/data-api/tacos?size=5"
```

Assuming there are more than five tacos to be seen, you can request the second page of tacos by adding the `page` parameter as follows:

```
$ curl "localhost:8080/data-api/tacos?size=5&page=1"
```

Notice that the `page` parameter is zero-based, which means that asking for page `1` is actually asking for the second page. (You'll also note that many command-line shells trip up over the ampersand in the request, which is why I quoted the whole URL in the preceding `curl` command.)

The `sort` parameter lets you sort the resulting list by any property of the entity. For example, you need a way to fetch the 12 most recently created tacos for the UI to display. You can do that by specifying the following mix of paging and sorting parameters:

```
$ curl "localhost:8080/data-api/tacos?sort=createdAt,desc&page=0&size=12"
```

Here the `sort` parameter specifies that you should sort by the `createdDate` property and that it should be sorted in descending order (so that the newest tacos are first). The `page` and `size` parameters specify that you should see the first page of 12 tacos.

This is precisely what the UI needs to show the most recently created tacos. It's approximately the same as the /api/tacos?recent endpoint you defined in `TacoController` earlier in this chapter.

Now let's switch gears and see how to write client code to consume the API endpoints we've created.

7.3 Consuming REST services

Have you ever gone to a movie and, as the movie starts, discovered that you were the only person in the theater? It certainly is a wonderful experience to have what is essentially a private viewing of a movie. You can pick whatever seat you want, talk back to the characters onscreen, and maybe even open your phone and tweet about it without anyone getting angry for disrupting their movie-watching experience. And the best part is that nobody else is there ruining the movie for you, either!

This hasn't happened to me often. But when it has, I have wondered what would have happened if I hadn't shown up. Would they still have shown the film? Would the hero still have saved the day? Would the theater staff still have cleaned the theater after the movie was over?

A movie without an audience is kind of like an API without a client. It's ready to accept and provide data, but if the API is never invoked, is it really an API? Like Schrödinger's cat, we can't know if the API is active or returning HTTP 404 responses until we issue a request to it.

It's not uncommon for Spring applications to both provide an API and make requests to another application's API. In fact, this is becoming prevalent in the world of microservices. Therefore, it's worthwhile to spend a moment looking at how to use Spring to interact with REST APIs.

A Spring application can consume a REST API with the following:

- `RestTemplate`—A straightforward, synchronous REST client provided by the core Spring Framework.
- *Traverson*—A wrapper around Spring's `RestTemplate`, provided by Spring HATEOAS, to enable a hyperlink-aware, synchronous REST client. Inspired from a JavaScript library of the same name.
- `WebClient`—A reactive, asynchronous REST client.

For now, we'll focus on creating clients with `RestTemplate`. I'll defer discussion of `WebClient` until we cover Spring's reactive web framework in chapter 12. And if you're interested in writing hyperlink-aware clients, check out the Traverson documentation at **http://mng.bz/aZno**.

There's a lot that goes into interacting with a REST resource from the client's perspective—mostly tedium and boilerplate. Working with low-level HTTP libraries, the client needs to create a client instance and a request object, execute the request, interpret the response, map the response to domain objects, and handle any exceptions that may be thrown along the way. And all of this boilerplate is repeated, regardless of what HTTP request is sent.

To avoid such boilerplate code, Spring provides `RestTemplate`. Just as `JdbcTemplate` handles the ugly parts of working with JDBC, `RestTemplate` frees you from dealing with the tedium of consuming REST resources.

`RestTemplate` provides 41 methods for interacting with REST resources. Rather than examine all of the methods that it offers, it's easier to consider only a dozen unique operations, each overloaded to equal the complete set of 41 methods. The 12 operations are described in table 7.2.

Table 7.2 `RestTemplate` defines 12 unique operations, each of which is overloaded, providing a total of 41 methods.

Method	Description
delete(...)	Performs an HTTP `DELETE` request on a resource at a specified URL
exchange(...)	Executes a specified HTTP method against a URL, returning a `ResponseEntity` containing an object mapped from the response body
execute(...)	Executes a specified HTTP method against a URL, returning an object mapped from the response body
getForEntity(...)	Sends an HTTP `GET` request, returning a `ResponseEntity` containing an object mapped from the response body
getForObject(...)	Sends an HTTP `GET` request, returning an object mapped from a response body
headForHeaders(...)	Sends an HTTP `HEAD` request, returning the HTTP headers for the specified resource URL
optionsForAllow(...)	Sends an HTTP `OPTIONS` request, returning the `Allow` header for the specified URL
patchForObject(...)	Sends an HTTP `PATCH` request, returning the resulting object mapped from the response body
postForEntity(...)	`POST` s data to a URL, returning a `ResponseEntity` containing an object mapped from the response body
postForLocation(...)	`POST` s data to a URL, returning the URL of the newly created resource
postForObject(...)	`POST` s data to a URL, returning an object mapped from the response body
put(...)	`PUT` s resource data to the specified URL

With the exception of `TRACE`, `RestTemplate` has at least one method for each of the standard HTTP methods. In addition, `execute()` and `exchange()` provide lower-level, general-purpose methods for sending requests with any HTTP method.

Most of the methods in table 7.2 are overloaded into the following three method forms:

- One accepts a `String` URL specification with URL parameters specified in a variable argument list.
- One accepts a `String` URL specification with URL parameters specified in a `Map<String,String>`.
- One accepts a `java.net.URI` as the URL specification, with no support for parameterized URLs.

Once you get to know the 12 operations provided by `RestTemplate` and how each of the variant forms works, you'll be well on your way to writing resource-consuming REST clients.

To use `RestTemplate`, you'll either need to create an instance at the point you need it, as follows:

```
RestTemplate rest = new RestTemplate();
```

or you can declare it as a bean and inject it where you need it, as shown next:

```
@Bean
public RestTemplate restTemplate() {
 return new RestTemplate();
}
```

Let's survey `RestTemplate`'s operations by looking at those that support the four primary HTTP methods: `GET`, `PUT`, `DELETE`, and `POST`. We'll start with `getForObject()` and `getForEntity()`—the `GET` methods.

3.1 GETting resources

Suppose that you want to fetch an ingredient from the Taco Cloud API. For that, you can use `RestTemplate`'s `getForObject()` to fetch the ingredient. For example, the following code uses `RestTemplate` to fetch an `Ingredient` object by its ID:

```
public Ingredient getIngredientById(String ingredientId) {
  return rest.getForObject("http://localhost:8080/ingredients/{id}",
                           Ingredient.class, ingredientId);
}
```

Here you're using the `getForObject()` variant that accepts a `String` URL and uses a variable list for URL variables. The `ingredientId` parameter passed into `getForObject()` is used to fill in the `{id}` placeholder in the given URL. Although there's only one URL variable in this example, it's important to know that the variable parameters are assigned to the placeholders in the order that they're given.

The second parameter to `getForObject()` is the type that the response should be bound to. In this case, the response data (that's likely in JSON format) should be deserialized into an `Ingredient` object that will be returned.

Alternatively, you can use a `Map` to specify the URL variables, as shown next:

```
public Ingredient getIngredientById(String ingredientId) {
  Map<String, String> urlVariables = new HashMap<>();
  urlVariables.put("id", ingredientId);
  return rest.getForObject("http://localhost:8080/ingredients/{id}",
     Ingredient.class, urlVariables);
}
```

In this case, the value of `ingredientId` is mapped to a key of `id`. When the request is made, the `{id}` placeholder is replaced by the map entry whose key is `id`.

Using a `URI` parameter is a bit more involved, requiring that you construct a `URI` object before calling `getForObject()`. Otherwise, it's similar to both of the other variants, as shown here:

```
public Ingredient getIngredientById(String ingredientId) {
  Map<String, String> urlVariables = new HashMap<>();
  urlVariables.put("id", ingredientId);
  URI url = UriComponentsBuilder
        .fromHttpUrl("http://localhost:8080/ingredients/{id}")
        .build(urlVariables);
  return rest.getForObject(url, Ingredient.class);
}
```

Here the `URI` object is defined from a `String` specification, and its placeholders filled in from entries in a `Map`, much like the previous variant of `getForObject()`. The `getForObject()` method is a no-nonsense way of fetching a resource. But if the client needs more than the payload body, you may want to consider using `getForEntity()`.

`getForEntity()` works in much the same way as `getForObject()`, but instead of returning a domain object that represents the response's payload, it returns a `ResponseEntity` object that wraps that domain object. The `ResponseEntity` gives access to additional response details, such as the response headers.

For example, suppose that in addition to the ingredient data, you want to inspect the `Date` header from the response. With `getForEntity()` that becomes straightforward, as shown in the following code:

```
public Ingredient getIngredientById(String ingredientId) {
  ResponseEntity<Ingredient> responseEntity =
    rest.getForEntity("http://localhost:8080/ingredients/{id}",
        Ingredient.class, ingredientId);
  log.info("Fetched time: {}",
        responseEntity.getHeaders().getDate());
  return responseEntity.getBody();
}
```

The `getForEntity()` method is overloaded with the same parameters as `getForObject()`, so you can provide the URL variables as a variable list parameter or call `getForEntity()` with a `URI` object.

3.2 PUTting resources

For sending HTTP PUT requests, `RestTemplate` offers the `put()` method. All three overloaded variants of `put()` accept an `Object` that is to be serialized and sent to the given URL. As for the URL itself, it can be specified as a `URI` object or as a `String`. And like `getForObject()` and `getForEntity()`, the URL variables can be provided as either a variable argument list or as a `Map`.

Suppose that you want to replace an ingredient resource with the data from a new `Ingredient` object. The following code should do the trick:

```
public void updateIngredient(Ingredient ingredient) {
  rest.put("http://localhost:8080/ingredients/{id}",
```

```
    ingredient, ingredient.getId());
}
```

Here the URL is given as a `String` and has a placeholder that's substituted by the given `Ingredient` object's `id` property. The data to be sent is the `Ingredient` object itself. The `put()` method returns `void`, so there's nothing you need to do to handle a return value.

3.3 DELETEing resources

Suppose that Taco Cloud no longer offers an ingredient and wants it completely removed as an option. To make that happen, you can call the `delete()` method from `RestTemplate` as follows:

```
public void deleteIngredient(Ingredient ingredient) {
  rest.delete("http://localhost:8080/ingredients/{id}",
    ingredient.getId());
}
```

In this example, only the URL (specified as a `String`) and a URL variable value are given to `delete()`. But as with the other `RestTemplate` methods, the URL could be specified as a `URI` object or the URL parameters given as a `Map`.

3.4 POSTing resource data

Now let's say that you add a new ingredient to the Taco Cloud menu. An HTTP `POST` request to the .../ingredients endpoint with ingredient data in the request body will make that happen. `RestTemplate` has three ways of sending a `POST` request, each of which has the same overloaded variants for specifying the URL. If you wanted to receive the newly created `Ingredient` resource after the `POST` request, you'd use `postForObject()` like this:

```
public Ingredient createIngredient(Ingredient ingredient) {
  return rest.postForObject("http://localhost:8080/ingredients",
    ingredient, Ingredient.class);
}
```

This variant of the `postForObject()` method takes a `String` URL specification, the object to be posted to the server, and the domain type that the response body should be bound to. Although you aren't taking advantage of it in this case, a fourth parameter could be a

`Map` of the URL variable value or a variable list of parameters to substitute into the URL.

If your client has more need for the location of the newly created resource, then you can call `postForLocation()` instead, as shown here:

```
public java.net.URI createIngredient(Ingredient ingredient) {
  return rest.postForLocation("http://localhost:8080/ingredients",
      ingredient);
}
```

Notice that `postForLocation()` works much like `postForObject()`, with the exception that it returns a `URI` of the newly created resource instead of the resource object itself. The `URI` returned is derived from the response's `Location` header. In the off chance that you need both the location and response payload, you can call `postForEntity()` like so:

```
public Ingredient createIngredient(Ingredient ingredient) {
  ResponseEntity<Ingredient> responseEntity =
        rest.postForEntity("http://localhost:8080/ingredients",
                           ingredient,
                           Ingredient.class);
  log.info("New resource created at {}",
          responseEntity.getHeaders().getLocation());
  return responseEntity.getBody();
}
```

Although the methods of `RestTemplate` differ in their purpose, they're quite similar in how they're used. This makes it easy to become proficient with `RestTemplate` and use it in your client code.

Summary

- REST endpoints can be created with Spring MVC, with controllers that follow the same programming model as browser-targeted controllers.
- Controller handler methods can either be annotated with `@ResponseBody` or return `ResponseEntity` objects to bypass the model and view and write data directly to the response body.
- The `@RestController` annotation simplifies REST controllers, eliminating the need to use `@ResponseBody` on handler methods.

- Spring Data repositories can automatically be exposed as REST APIs using Spring Data REST.

---

[1] Mapping HTTP methods to create, read, update, and delete (CRUD) operations isn't a perfect match, but in practice, that's how they're often used and how you'll use them in Taco Cloud.

Previous chapter
< Part 2. Integrated Spring

Next chapter
8 Securing REST >